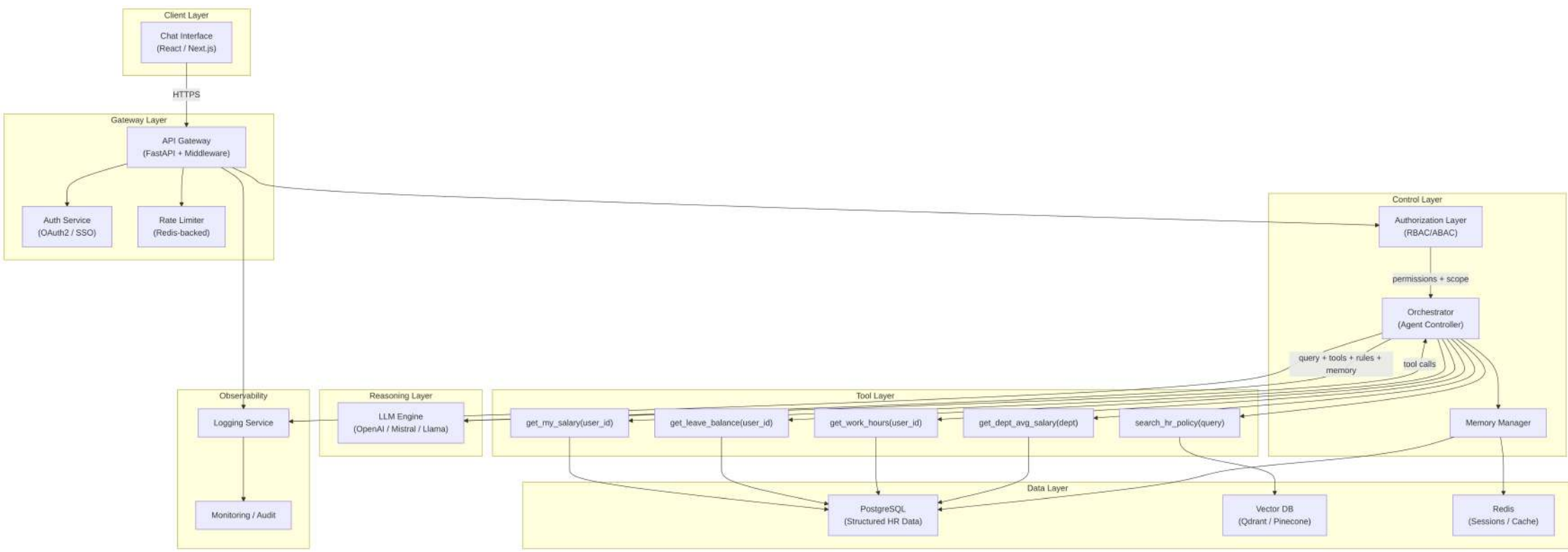


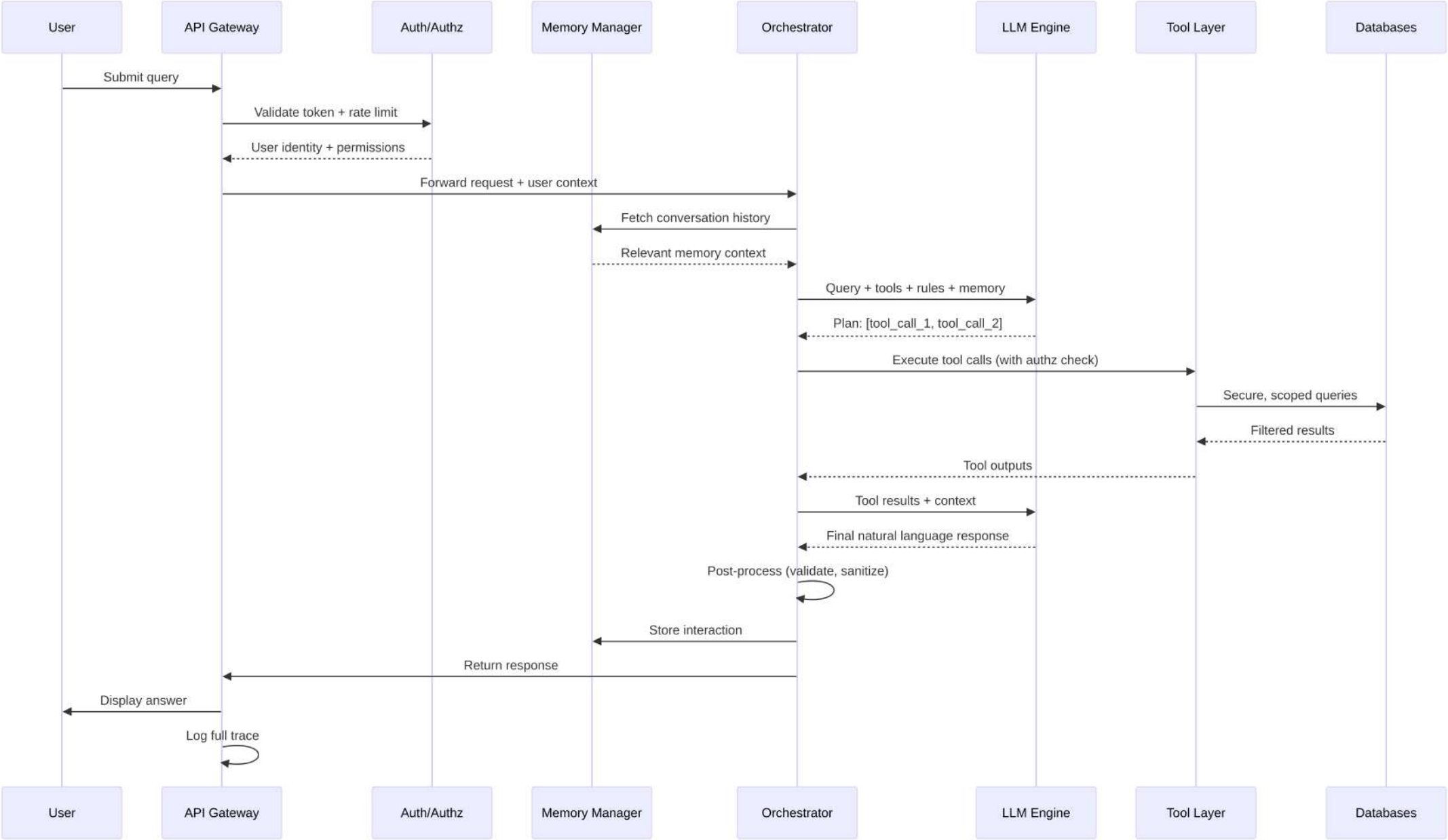
# AI-Powered HR Assistant — Complete Execution Plan

## Contents

|          |                                                             |          |
|----------|-------------------------------------------------------------|----------|
| <b>1</b> | <b>System Architecture</b>                                  | <b>2</b> |
| <b>2</b> | <b>Technology Stack Summary</b>                             | <b>5</b> |
| <b>3</b> | <b>Phases and Tasks</b>                                     | <b>5</b> |
| 3.1      | Phase 1: Infrastructure & Project Setup . . . . .           | 5        |
| 3.2      | Phase 2: Data Layer . . . . .                               | 6        |
| 3.3      | Phase 3: Core Backend — Auth & API Gateway . . . . .        | 6        |
| 3.4      | Phase 4: LLM Integration & Orchestration . . . . .          | 6        |
| 3.5      | Phase 5: Memory & Conversation Management . . . . .         | 7        |
| 3.6      | Phase 6: Frontend — Chat Interface . . . . .                | 7        |
| 3.7      | Phase 7: Testing, Security Hardening & Deployment . . . . . | 7        |
| <b>4</b> | <b>Potential Challenges &amp; Solutions</b>                 | <b>7</b> |
| <b>5</b> | <b>Timeline Estimate</b>                                    | <b>8</b> |
| <b>6</b> | <b>Finalized Decisions</b>                                  | <b>8</b> |

# 1 System Architecture





## 2 Technology Stack Summary

| Component        | Technology                                             | Justification                                |
|------------------|--------------------------------------------------------|----------------------------------------------|
| Backend API      | Python + FastAPI                                       | Async-native, OpenAPI docs, high performance |
| LLM Integration  | LangChain / LlamaIndex                                 | Tool-use orchestration, memory abstractions  |
| LLM Provider     | OpenAI GPT-4o / Mistral / Llama 3                      | Function-calling support, strong reasoning   |
| Structured DB    | PostgreSQL 16                                          | Mature, RBAC-friendly, JSON support          |
| Vector DB        | Qdrant (self-hosted) or Pinecone (managed)             | Semantic search for RAG                      |
| Embedding Model  | OpenAI text-embedding-3-small or sentence-transformers | Document vectorization                       |
| Cache / Sessions | Redis                                                  | Fast KV store, rate limiting, session data   |
| Auth             | OAuth2 + JWT                                           | Industry standard, SSO-compatible            |
| Frontend         | React + Next.js                                        | Responsive chat UI                           |
| Containerization | Docker + Docker Compose                                | Reproducible local dev & deployment          |
| Monitoring       | Structured logging + Prometheus + Grafana              | Observability                                |
| Document Parsing | unstructured / PyMuPDF                                 | PDF/DOCX → text for embedding                |

## 3 Phases and Tasks

### 3.1 Phase 1: Infrastructure & Project Setup

**Objectives:** Set up project repo, dev environment, CI, coding standards, and containerization.

**Tasks:**

1. Initialize project repository with monorepo structure (backend, frontend, docs, scripts, docker-compose.yml, .env.example)
2. Set up Python virtual environment and install dependencies (FastAPI, SQLAlchemy, LangChain, OpenAI, Redis, etc.)
3. Configure Docker Compose for local services (PostgreSQL, Redis, Qdrant, backend)
4. Environment configuration with ‘.env’ and Pydantic ‘Settings’ class

**Expected Outputs:** Runnable project skeleton, all services reachable, FastAPI health endpoint returns 200 OK.

### 3.2 Phase 2: Data Layer

**Objectives:** Design and implement PostgreSQL schema, build RAG ingestion pipeline.

**Tasks:**

1. Design PostgreSQL schema: employees, departments, salaries, leave balances, work hours, users, conversations, messages, audit\_logs
2. Create SQLAlchemy models
3. Set up Alembic migrations
4. Seed sample data
5. Create document ingestion script for HR policies, chunking, embeddings, store in Qdrant
6. Build retrieval function: `search_hr_policy(query)`

**Expected Outputs:** Database schema with seed data, vector DB populated, semantic search functional.

### 3.3 Phase 3: Core Backend — Auth & API Gateway

**Objectives:** Secure authentication, RBAC, API gateway middleware.

**Tasks:**

1. JWT-based auth flow
2. FastAPI auth dependency
3. Define roles and permissions (employee, manager, hr\_admin)
4. Implement authorization middleware
5. Tool-level authorization enforcement
6. Rate limiting, request logging, CORS configuration

**Expected Outputs:** Working auth, protected endpoints, role-based tool filtering, rate limiting active.

### 3.4 Phase 4: LLM Integration & Orchestration

**Objectives:** Orchestrator connects LLM to tools; define callable tools.

**Tasks:**

1. Define tools as functions with schemas

2. Implement all tools: `get_my_salary`, `get_leave_balance`, `get_work_hours`, `search_hr_policy`, etc.
3. Build Orchestrator class with agent loop, memory retrieval, post-processing, audit logging
4. Implement system prompt engineering
5. LLM provider abstraction layer

**Expected Outputs:** Orchestrator routes queries correctly, LLM uses tools, multi-step queries handled.

### 3.5 Phase 5: Memory & Conversation Management

**Objectives:** Short-term and long-term memory, multi-turn context.

**Tasks:** Short-term messages in PostgreSQL + Redis cache, long-term summaries with LLM, semantic retrieval optional, strict user isolation.

**Expected Outputs:** Context maintained, follow-ups resolved, summaries retrievable.

### 3.6 Phase 6: Frontend — Chat Interface

**Objectives:** Responsive chat UI.

**Tasks:** Create React/Next.js frontend, message streaming, tool indicators, auth flow, responsive design.

**Expected Outputs:** Working chat UI, streaming messages, history loads, auth functional.

### 3.7 Phase 7: Testing, Security Hardening & Deployment

**Objectives:** Test coverage, security, production deployment.

**Tasks:** Unit/integration/security/load tests, prompt injection defense, data encryption, TLS, containerized deployment, CI/CD, monitoring.

**Expected Outputs:** 80% test coverage, production-ready deployment, prompt injection handled.

## 4 Potential Challenges & Solutions

| Challenge         | Risk Level | Solution                                                                                 |
|-------------------|------------|------------------------------------------------------------------------------------------|
| Prompt injection  | High       | Multi-layer defense: input sanitization, system prompt, output validation, secondary LLM |
| LLM hallucination | High       | Force tool-use via function calling, validate answers                                    |

|                           |        |                                                             |
|---------------------------|--------|-------------------------------------------------------------|
| Token cost explosion      | Medium | Conversation summarization, context window management       |
| Latency                   | Medium | Parallel tool execution, Redis caching, streaming responses |
| Data staleness            | Medium | Short TTL caches, cache invalidation on updates             |
| Multi-tenant data leakage | High   | Enforce user_id filtering at every layer                    |
| Memory bloat              | Medium | Automatic summarization, retention policies, pagination     |
| LLM provider outages      | Medium | Provider abstraction with fallback                          |

## 5 Timeline Estimate

| Phase                           | Duration        | Dependencies |
|---------------------------------|-----------------|--------------|
| Phase 1: Infrastructure & Setup | 1 week          | None         |
| Phase 2: Data Layer             | 1.5 weeks       | Phase 1      |
| Phase 3: Auth & Authorization   | 1 week          | Phase 1      |
| Phase 4: LLM & Orchestration    | 2 weeks         | Phase 2,3    |
| Phase 5: Memory Management      | 1 week          | Phase 2,4    |
| Phase 6: Frontend               | 1.5 weeks       | Phase 3,4    |
| Phase 7: Testing & Deployment   | 1.5 weeks       | All phases   |
| <b>Total</b>                    | <b>10 weeks</b> |              |

## 6 Finalized Decisions

| Decision        | Choice                                                                   |
|-----------------|--------------------------------------------------------------------------|
| LLM Provider    | Groq API (fast inference, OpenAI-compatible, supports Llama 3 / Mixtral) |
| Vector Database | ChromaDB (lightweight, embedded)                                         |
| Frontend        | Next.js (full multi-page application)                                    |
| Deployment      | Docker Compose (all services containerized)                              |
| Scope           | Full project — all 7 phases                                              |